

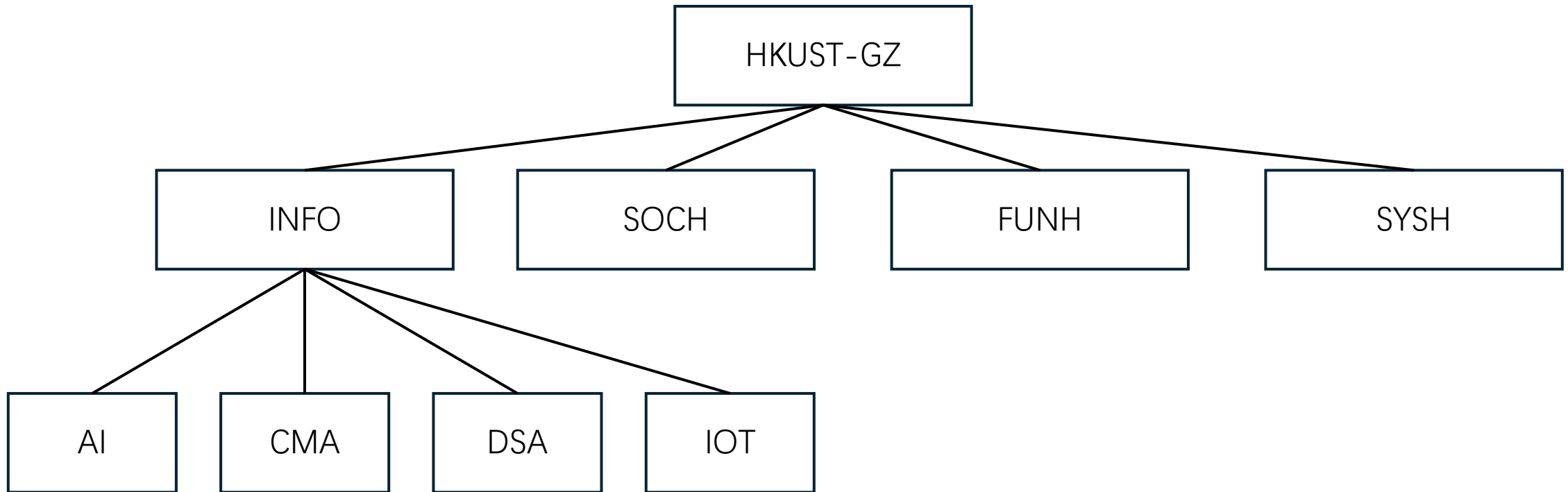
UFUG 2602

Tree basics and traversal

Jia LI & Shangqi LU

Data Science and Analytics Thrust
Hong Kong University of Science and Technology (Guangzhou)

Example 1: HKUST(GZ)



Example 2: File System

- In each directory, we can create new directories or files.

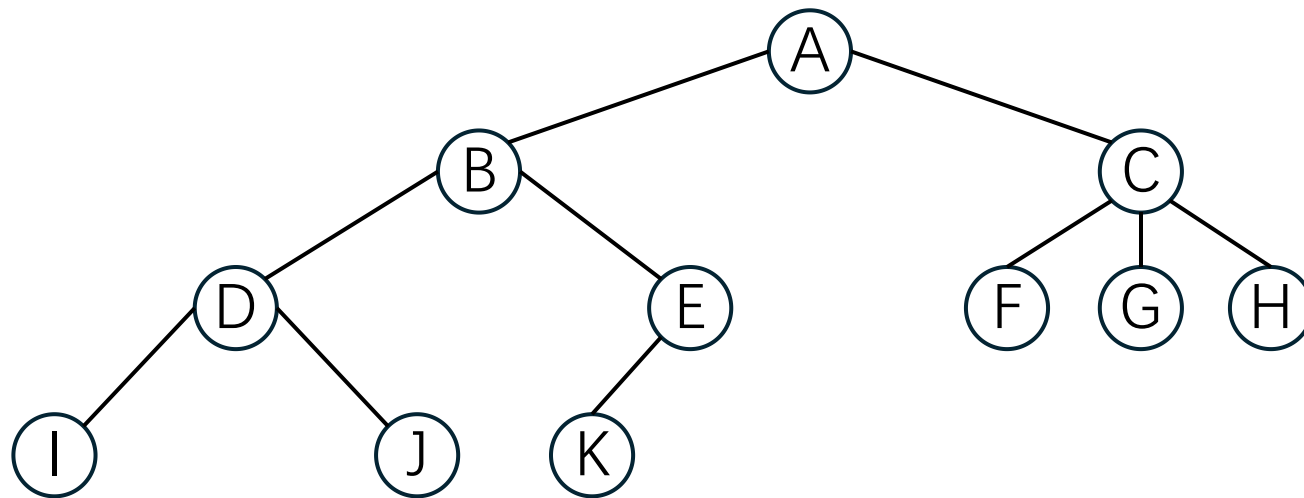
```
|---- | datasets/  
|---- | --- | ImageNet-1k/  
|---- | --- | --- | train_dataset/  
|---- | --- | --- | ---0.jpg  
|---- | --- | --- | ---1.jpg  
|---- | --- | --- | ---...  
|---- | --- | --- | eval_dataset/  
|---- | --- | --- | test_dataset/  
|---- | --- | --- | train_data.json  
|---- | --- | --- | eval_data.json  
|---- | --- | --- | test_data.json  
|---- | --- | ...  
|---- | --- | ESC-50/  
|---- | --- | ...
```

Outline

- **Tree Concept**
- Binary Tree
- Tree Traversal

Introduction

- A data structure is considered **linear** if its elements form a sequence.
 - E.g., arrays, linked lists, stacks and queues.
- A **tree** data structure is a collection of nodes connected by directed edges.
- It is a **non-linear structure** that organizes data into a hierarchy.

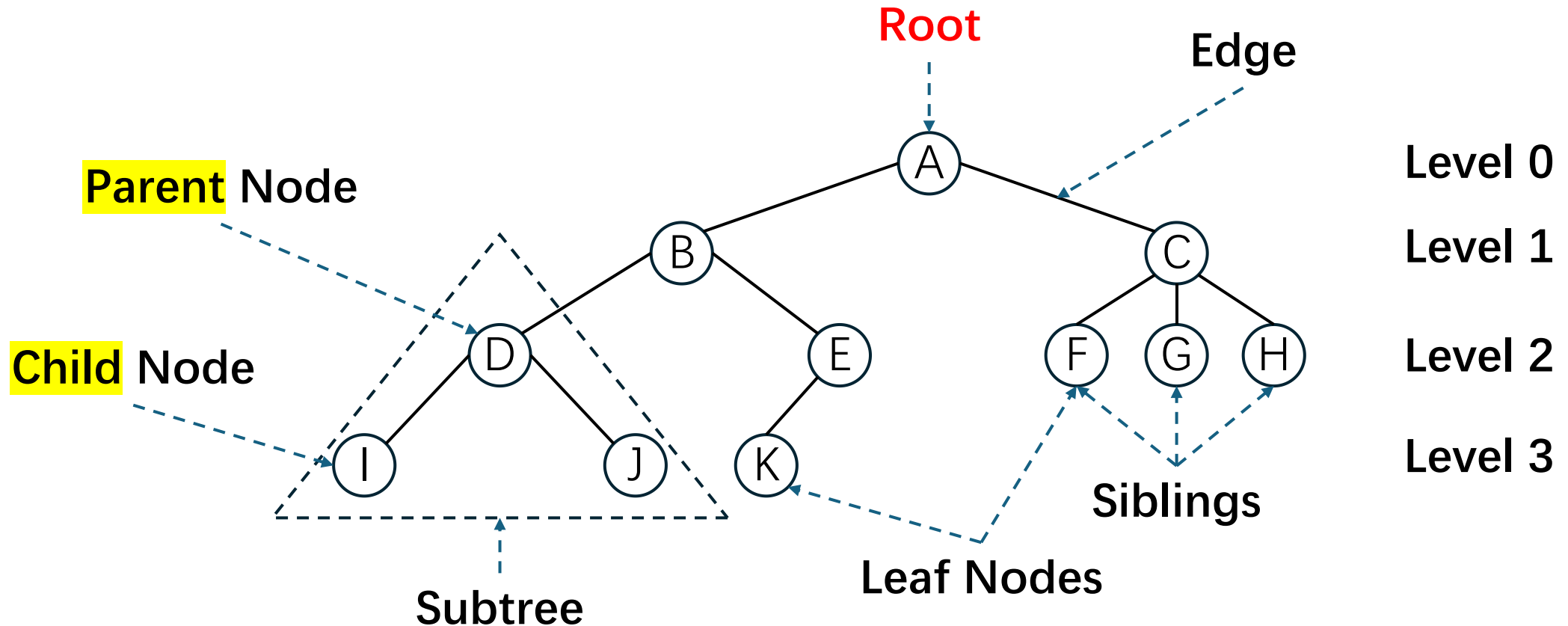


Definition

- A tree T is a finite set of one or more nodes such that
 - There is a special node called the **root**.
 - The remaining nodes are partitioned into $b \geq 0$ disjoint sets $T_1, \dots, T_b$ where each of these sets is itself a tree.
 - We call $T_1, \dots, T_b$ the **subtrees** of the root.
 - The root of T connects to the roots of all its subtrees.

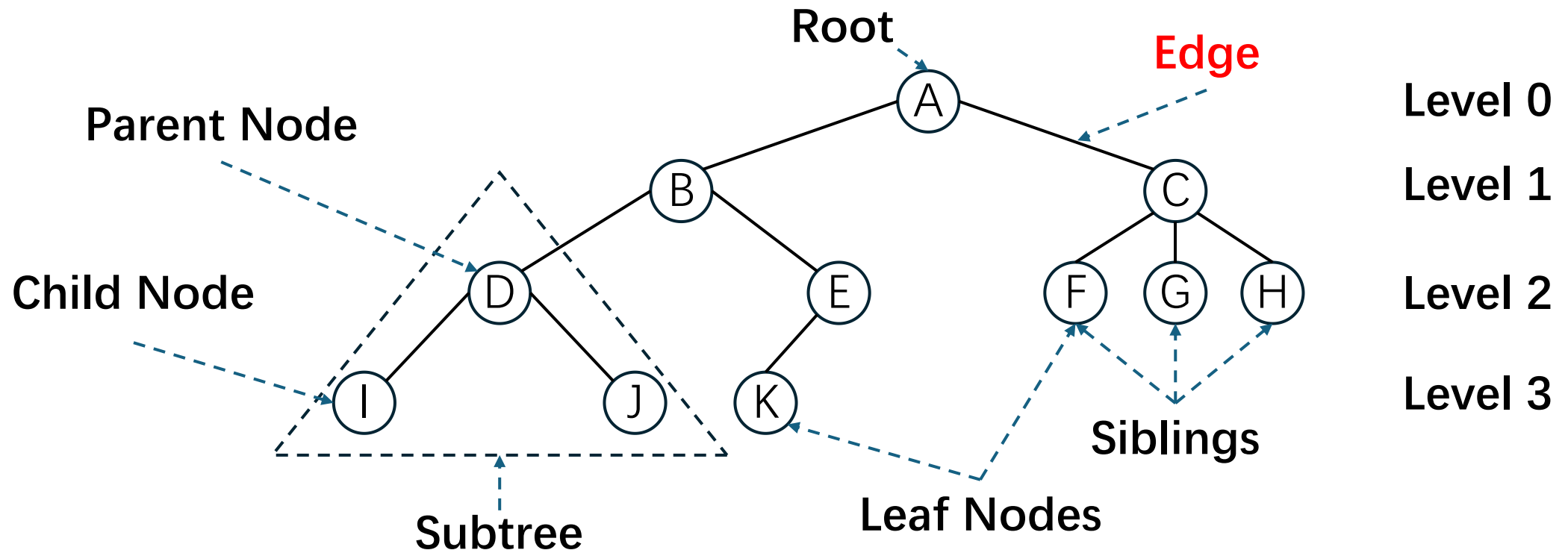
Definition

- The top-most node of the tree is called the **root**.
- Every tree has one and only one root node.



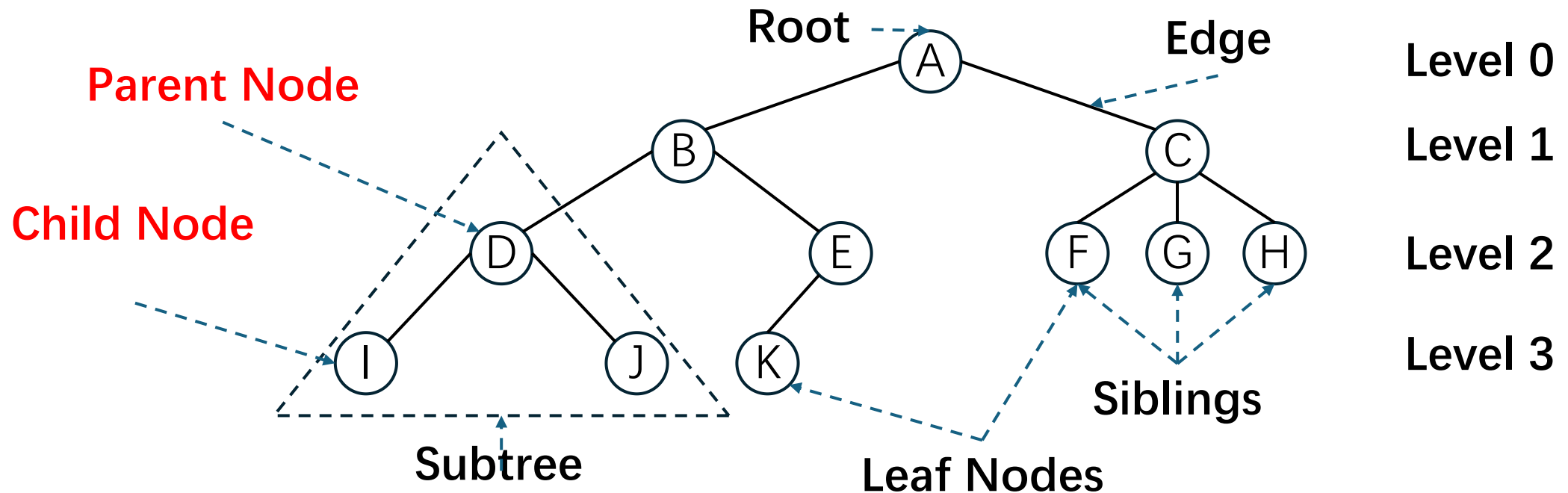
Definition

- The connections between nodes are called **edges**.
- For each edge, we assume the higher node points to the lower node.



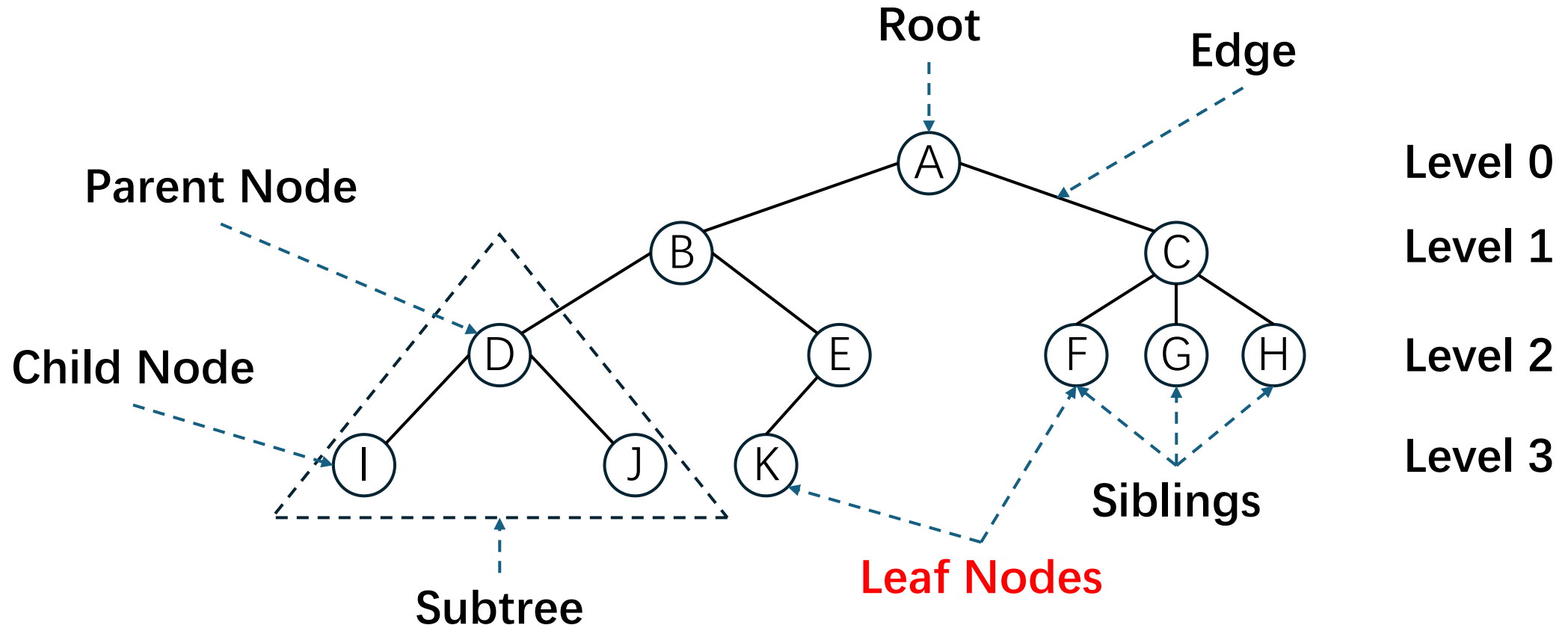
Definition

- For each edge pointing from **x** to **y**, we say
 - x is the **parent** of y (each node has at most one parent);
 - y is the **child** of x (each node may have more than one child nodes).
- Each node (excluding root) has exactly one parent and root has no parent.



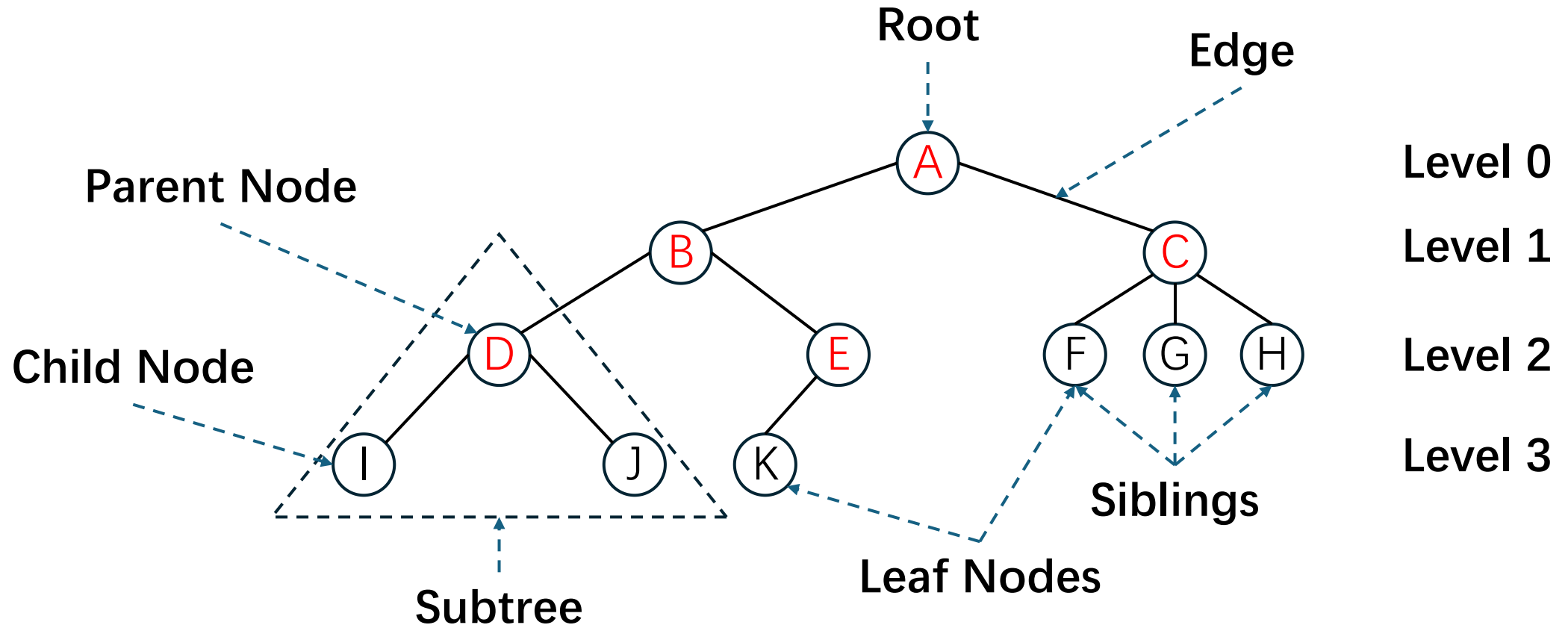
Definition

- **Leaf** node: The nodes which do not have any child nodes are called leaf nodes. I, J, K, F, G and H are the leaf nodes of the tree.



Definition

- **Internal** node: A node with at least one child is called an internal node.

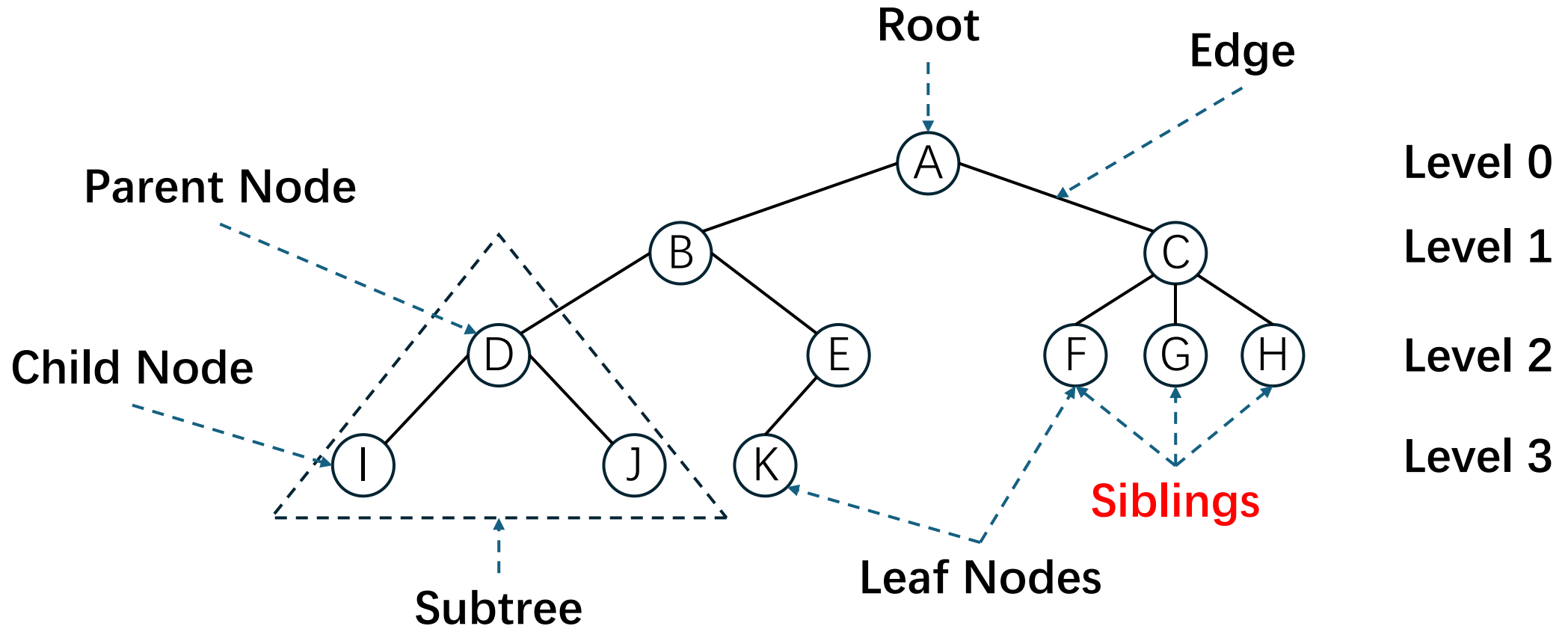


Definition

- **Ancestor** of a node x : any node on the path from the root to x is an ancestor of x . A and B are the ancestor nodes of E .
- Note: A node is an ancestor of itself.
- x is an ancestor of y is equivalent to y is a **descendant** of x
- In particular, if $x \neq y$, we say that x is a **proper** ancestor of y , and likewise, y is a **proper** descendant of x .

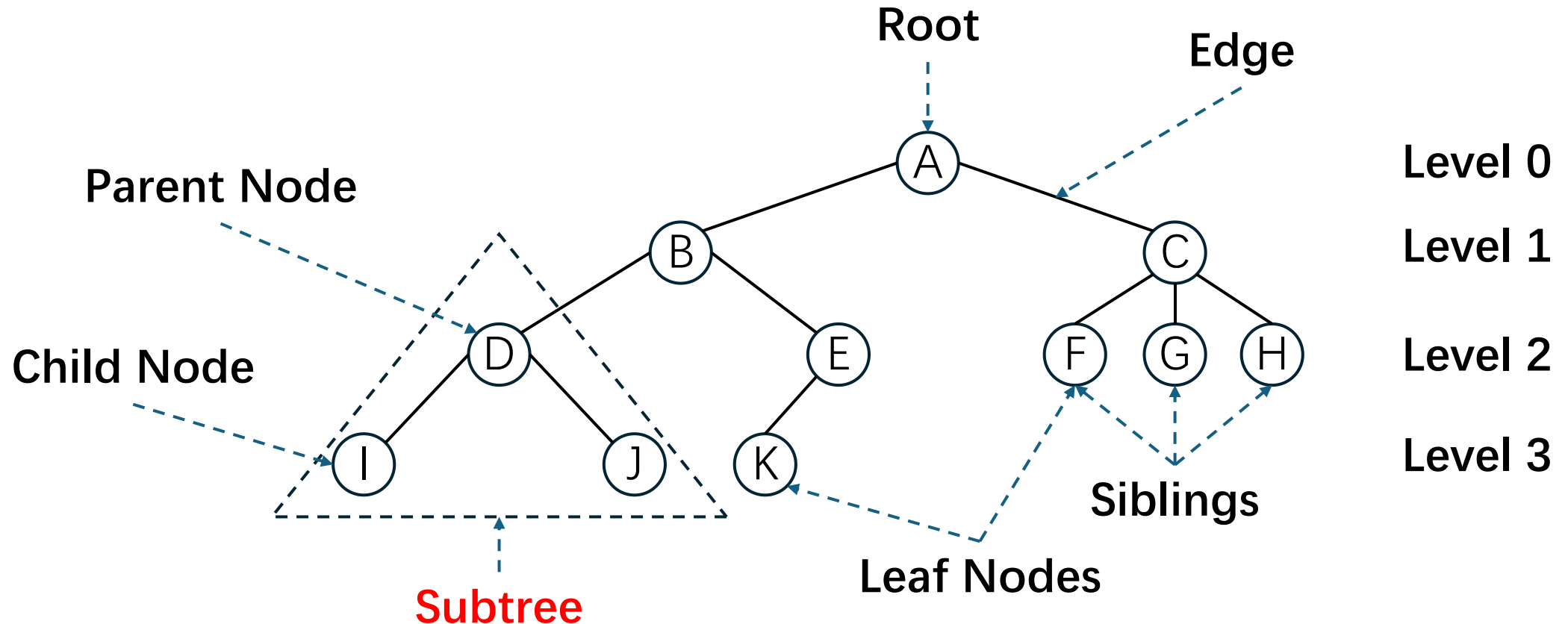
Definition

- **Sibling**: Children of the same parent node are called siblings. F,G, and H are called siblings.



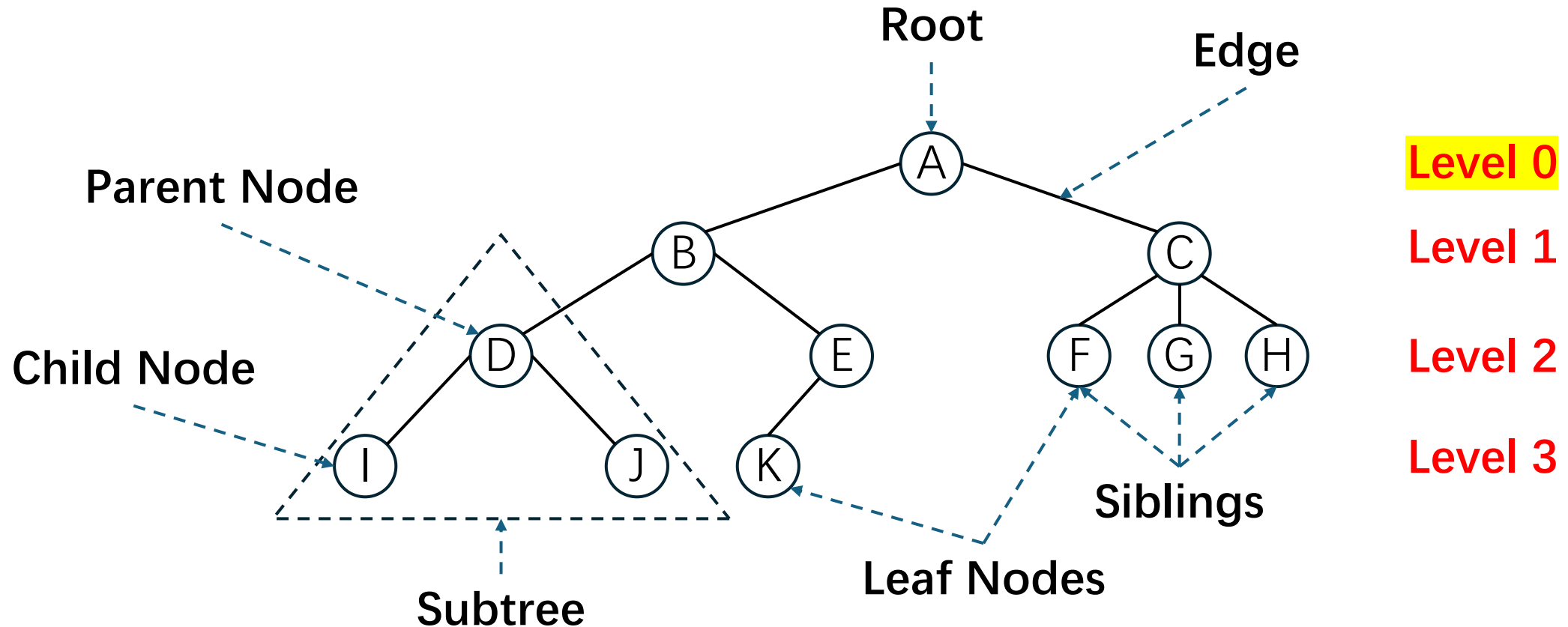
Definition

- **Subtree**: given any node u , the subtree of u consists of all the descendants of u , along with the edges that connect them.



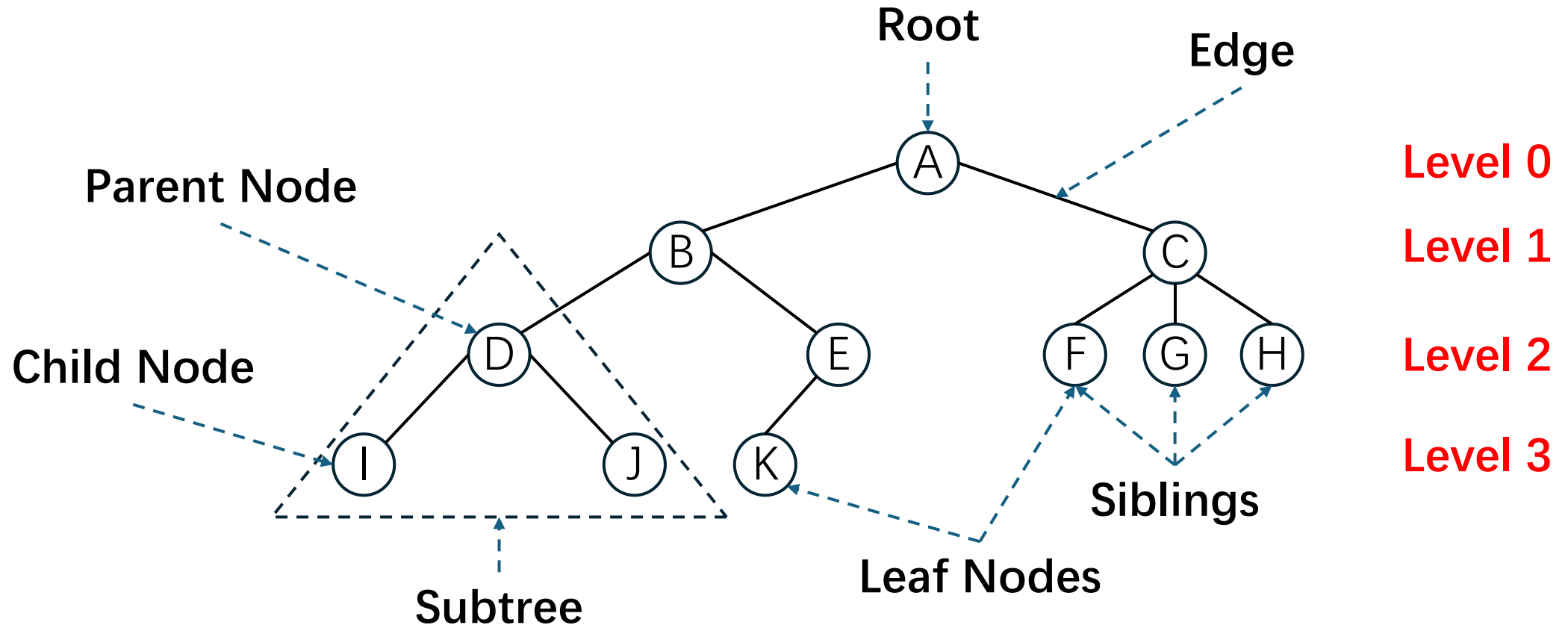
Definition

- **Level** of a node: The number of edges on the path from root to that node. The root node has level 0.



Definition

- **Height:** The height of a tree is the length of the longest path from the root node to a leaf node.

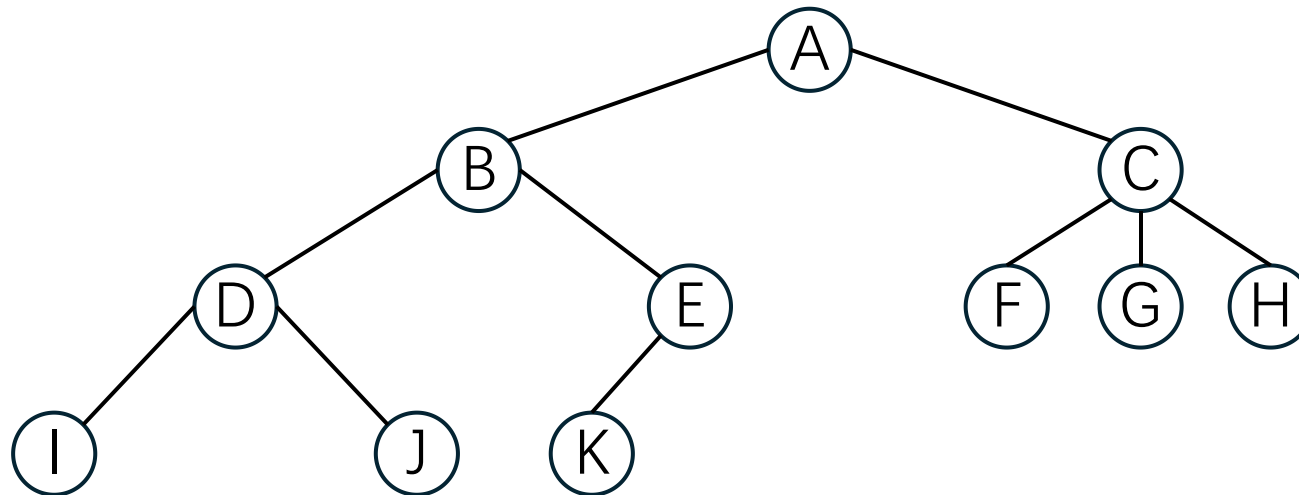


Implementation

```
class Node:  
    def __init__(self, data):  
        self.data = data  
        self.children = [] # a list of nodes
```

Property

- **Lemma:** A tree with n nodes has $n - 1$ edges.
- **Proof:** Every node (except the root) has an edge leading to it. Then, the number of edges = the number of nodes $- 1$.
- Hence, number of nodes = number of edges $+ 1$.

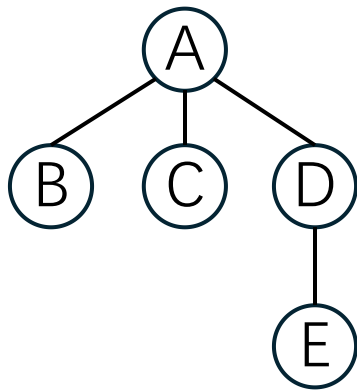


Outline

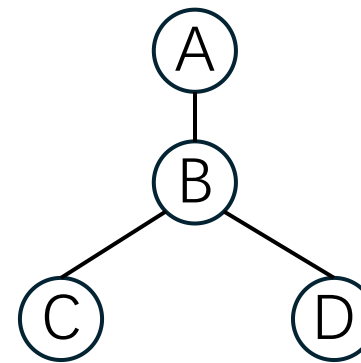
- Tree Concept
- **Binary Tree**
- Tree Traversal

k-Ary and Binary Tree

- A **k-ary** tree is a rooted tree where every internal node has at most k child nodes.
- A 2-ary tree is called a **binary tree**.



3-ary tree



Binary tree

Binary Tree — Left and Right

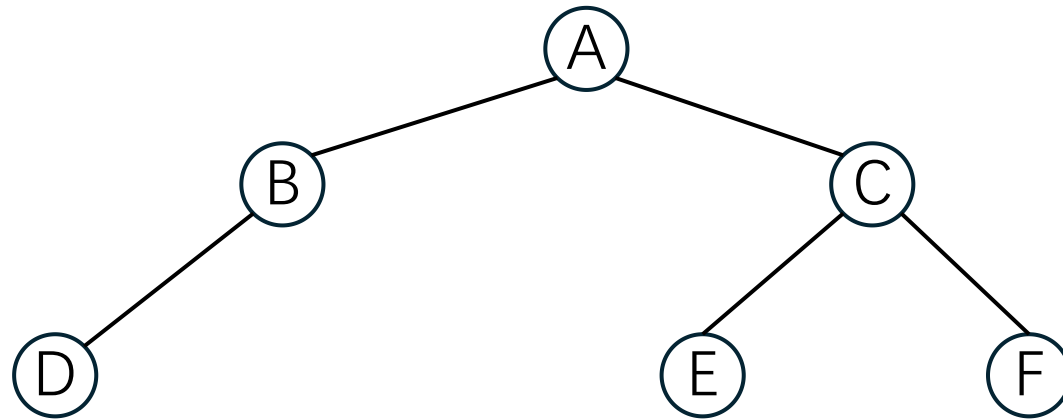
- A binary tree is **left-right labeled** if
 - Every node **v** — except the root — has been designated either as a **left** or **right** node of its parent.
 - Every internal node has at most one left child, and at most one right child.
- Throughout this course, we will discuss **only** binary trees that have been left-right labeled. Because of this, by a “binary tree”, we always refer to a left-right labeled one.

Binary Tree — Left and Right

- A (left-right labeled) binary tree implies an ordering among the nodes at the same level.
- Let u and v be nodes at the same level with parents p_u and p_v , respectively. We say that u is **on the left** of v if either of the following holds:
 - $p_u = p_v$ and u is the left child (implying that v is the right child);
 - $p_u \neq p_v$ and p_u is on the left of p_v .
- Accordingly, we say that v is **on the right** of u .

Binary Tree — Left and Right

- At Level 1, B is on the left of C.
- At Level 2, the nodes from left to right are D, E, and F.



Implementation

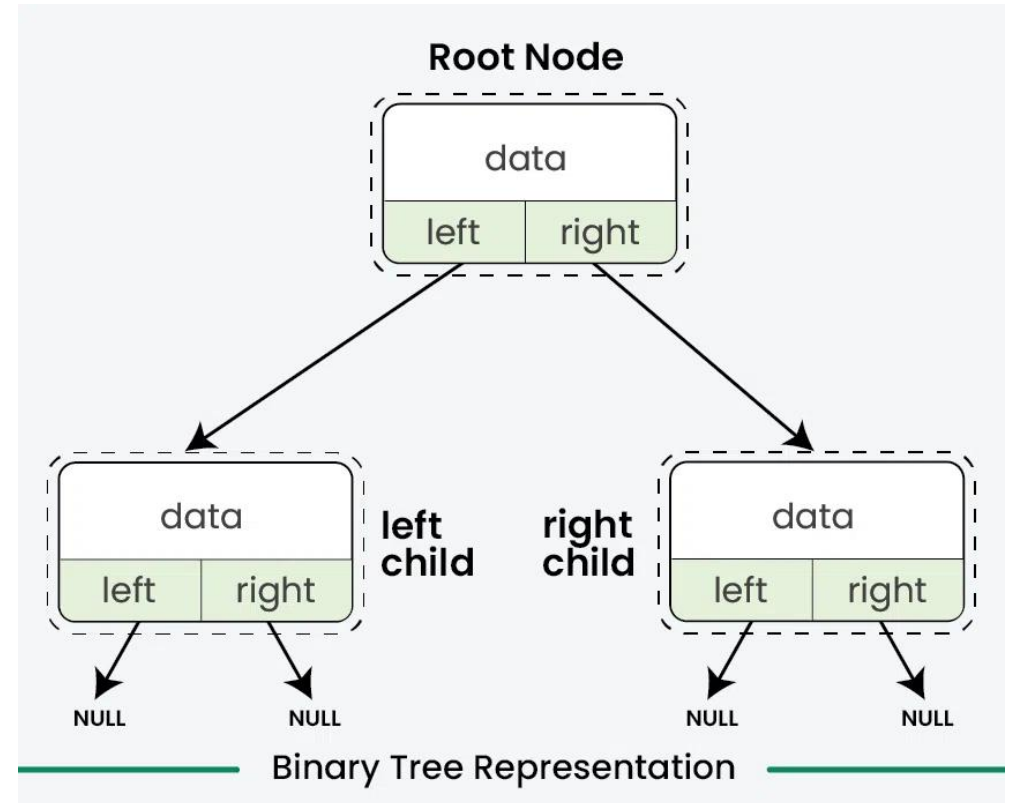
A Python class that represents
an individual node in a Binary Tree
class Node:

```
def __init__(self, data):
```

```
    self.left = None
```

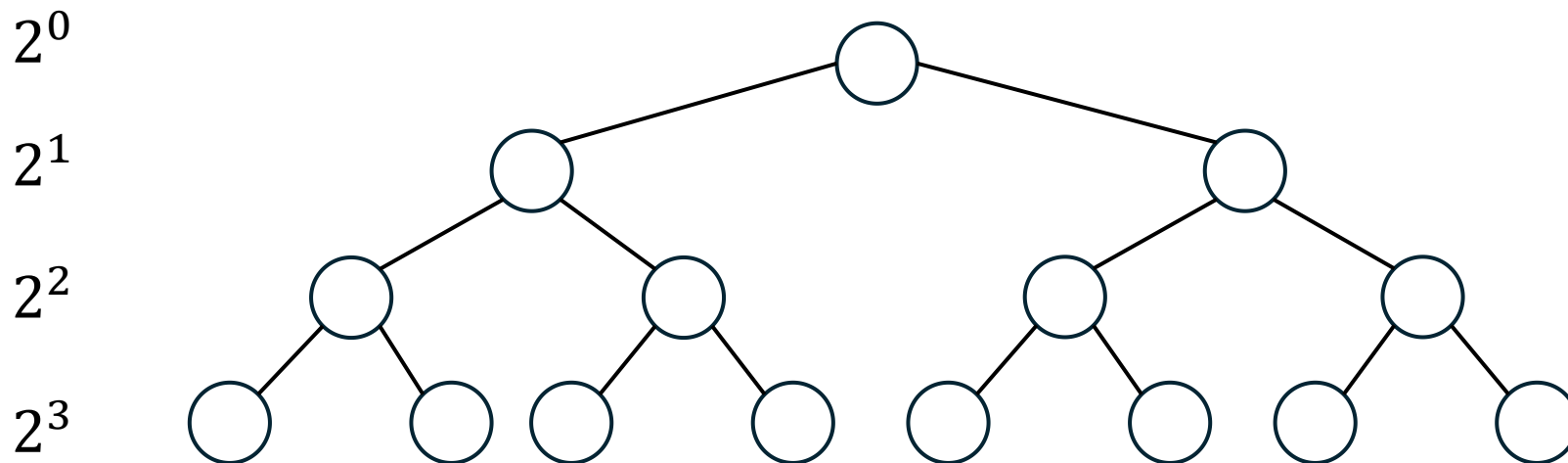
```
    self.right = None
```

```
    self.data = data
```



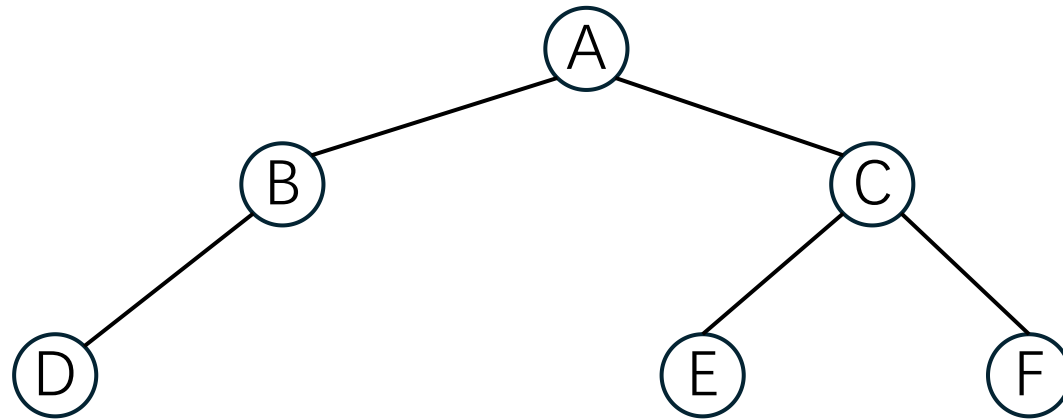
Binary Tree — A Property

- **Lemma:** The maximum number of nodes in a binary tree with k levels is $2^k - 1$ for $k \geq 0$.
- **Proof:** The maximum number of nodes on level i of a binary tree is 2^{i-1} .
- max # of nodes (level 1) + max # of node (level 2) + ... + max # of node (level k) = $\sum_{i=1}^k 2^{i-1} = 2^k - 1$.



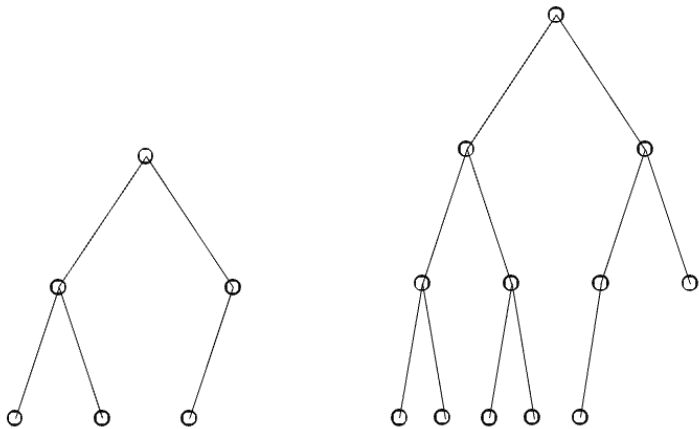
Binary Tree — Full Level

- Consider a binary tree with height h . Its level l ($0 \leq l \leq h$) is full if it contains 2^l nodes.
- Levels 0 and 1 are full, but level 2 is not.

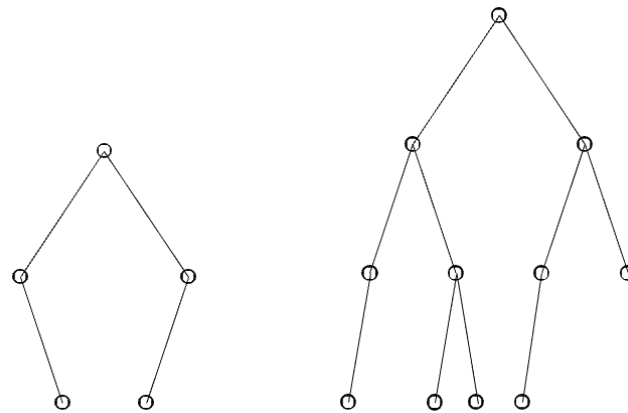


Complete Binary Tree 完全二叉树

- A binary tree is complete if all of its levels are completely filled, except possibly for the bottom-most level, which is filled from left to right without any gaps.
- 除了最后一层，其他所有层都必须是满的。
- 最后一层的节点都必须连续地靠左排列。



Complete Binary Trees



Not Complete Binary Trees

Complete Binary Tree — A Property

- **Lemma:** A complete binary tree with $n \geq 2$ nodes has height $O(\log n)$.
- **Proof:** Let h be the height of the binary tree. As Levels 0, 1, ..., $h - 1$ are full, we know that

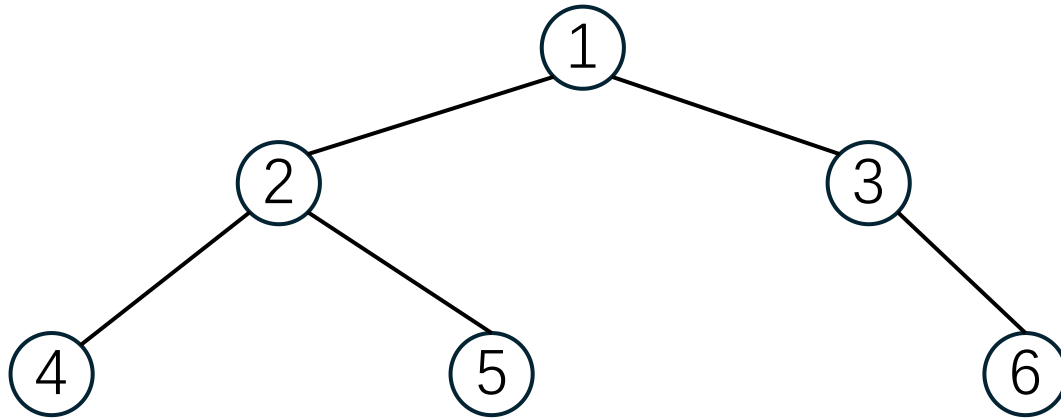
$$\begin{aligned} 2^0 + 2^1 + \dots + 2^{h-1} + 1 &\leq n \\ \Rightarrow 2^h &\leq n \\ \Rightarrow h &\leq \log_2 n = O(\log n). \end{aligned}$$

Outline

- Tree Concept
- Binary Tree
- **Tree Traversal**

Tree Traversals — Motivation

- A tree is a non-linear data structure without a fixed order.
- In many applications, it is essential to visit all the nodes within a tree.
- Given a binary tree T , how to visit each node exactly once?
 - How can we print all the nodes in T ?
 - How can we determine the number of leaf nodes in T ?

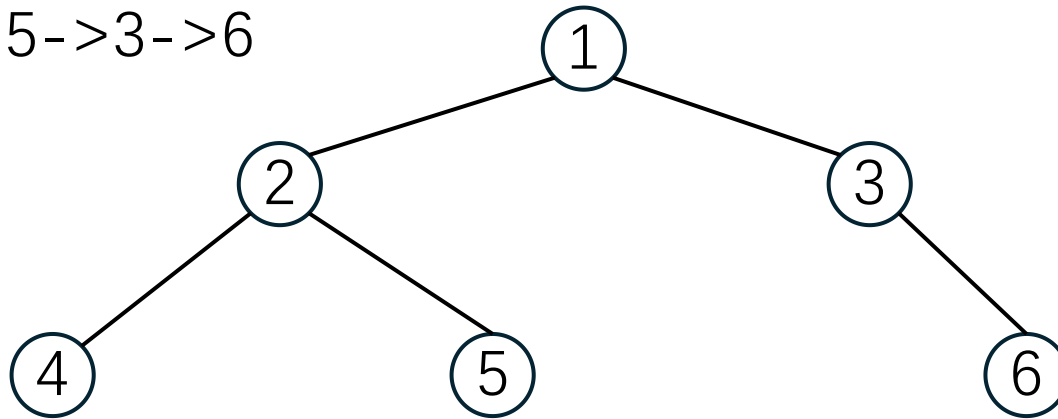


Tree Traversals — Overview

- Unlike linear data structures, which have a single logical traversal path, trees offer multiple traversal methods.
- These methods can yield different orders of node visitation, including:
- Depth-first traversal
 - Pre-order
 - In-order
 - Post-order
- Breadth-first traversal

Pre-order Traversal 前序遍历

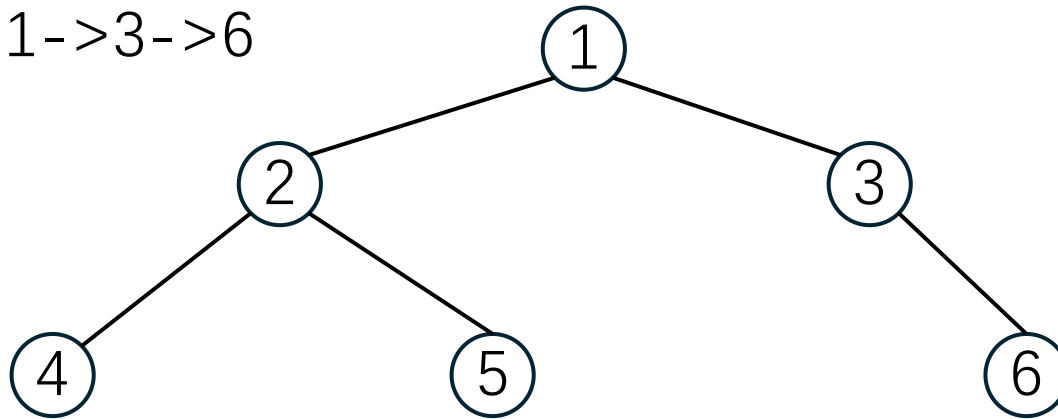
- Pre-order:
 1. Print the current node;
 2. Recursively traverse the current node's left subtree;
 3. Recursively traverse the current node's right subtree.
- Traversal Order:
 - 1->2->4->5->3->6



```
def pre_order(node):  
    if node is not None:  
        print(node.data)  
        pre_order(node.left)  
        pre_order(node.right)
```

In-order Traversal 中序遍历

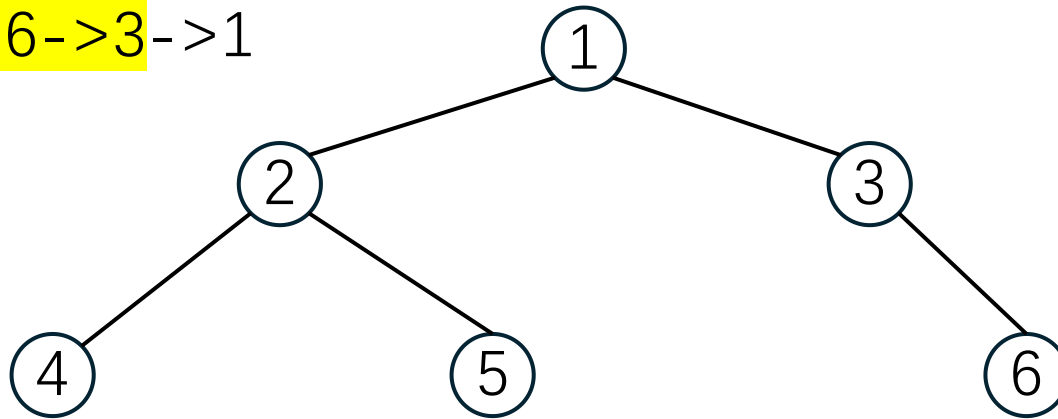
- In-order:
 1. Recursively traverse the current node's left subtree;
 2. Print the current node;
 3. Recursively traverse the current node's right subtree.
- Traversal Order:
 - 4->2->5->1->3->6



```
def in_order(node):  
    if node is not None:  
        in_order(node.left)  
        print(node.data)  
        in_order(node.right)
```

Post-order Traversal 后序遍历

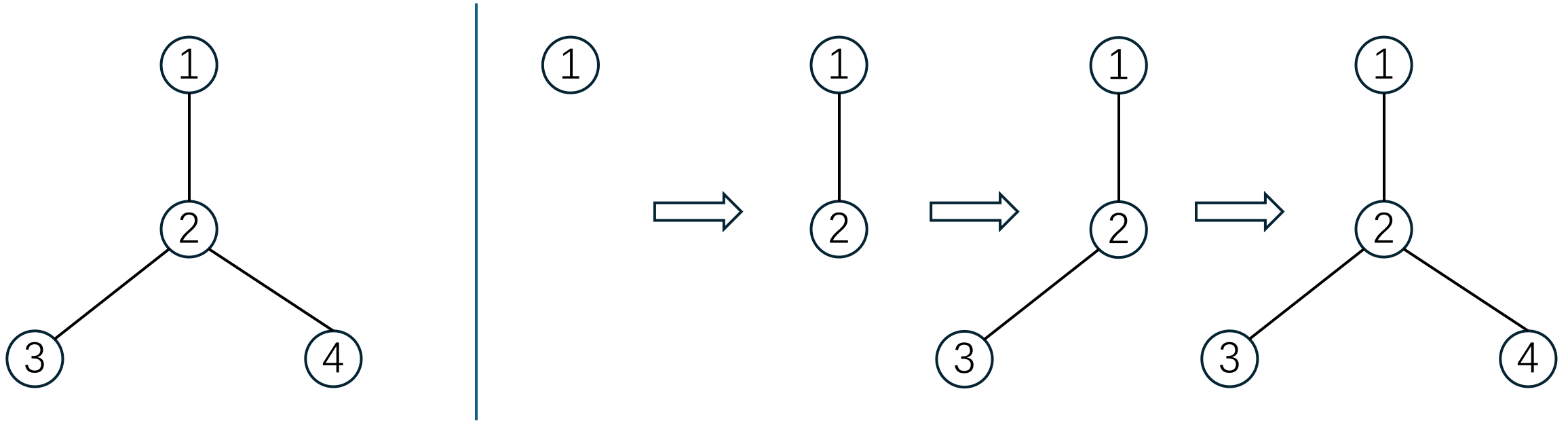
- Post-order:
 1. Recursively traverse the current node's left subtree;
 2. Recursively traverse the current node's right subtree;
 3. Print the current node.
- Traversal Order:
 - 4->5->2->6->3->1



```
def post_order(node):  
    if node is not None:  
        post_order(node.left)  
        post_order(node.right)  
        print(node.data)
```

Application — Tree Copy

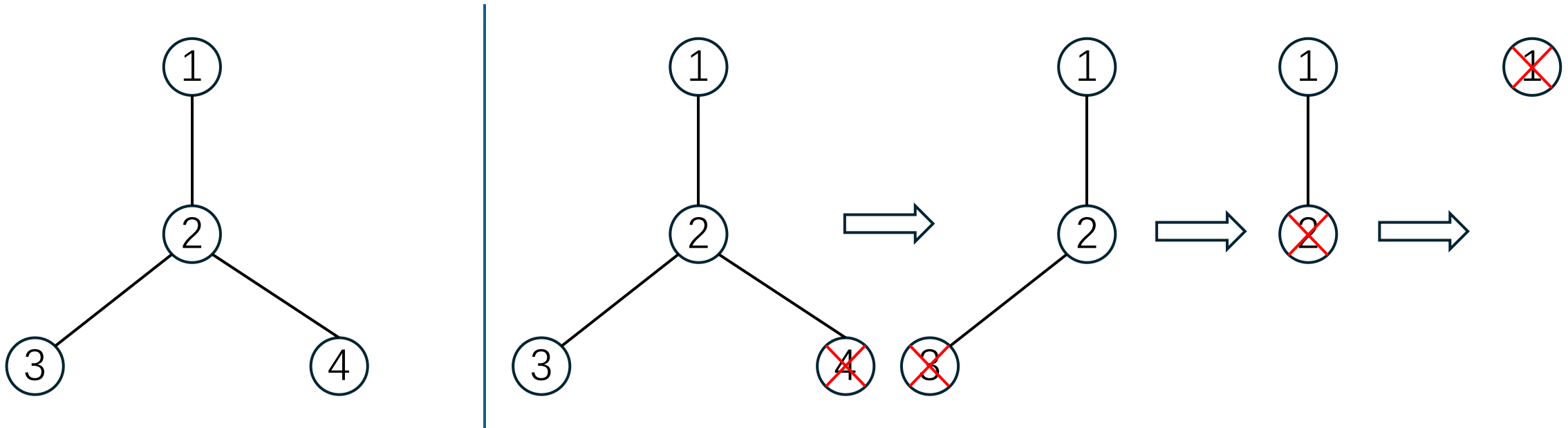
- Depending on the specific application requirements, we choose the most appropriate traversal order to solve the problem.
- Creating/copying a tree is constructed from parent to child. Therefore, it's better to choose pre-order.



- # 使用前序遍历模式复制树的简单示例
- def copy_tree(original_node):
 - if original_node is None:
 - return None
 - # 1. 处理根节点： 首先创建新的父节点
 - new_node = Node(original_node.data)
 - # 2. 访问左子树： 递归复制左子树并连接
 - new_node.left = copy_tree(original_node.left)
 - # 3. 访问右子树： 递归复制右子树并连接
 - new_node.right = copy_tree(original_node.right)
 - return new_node

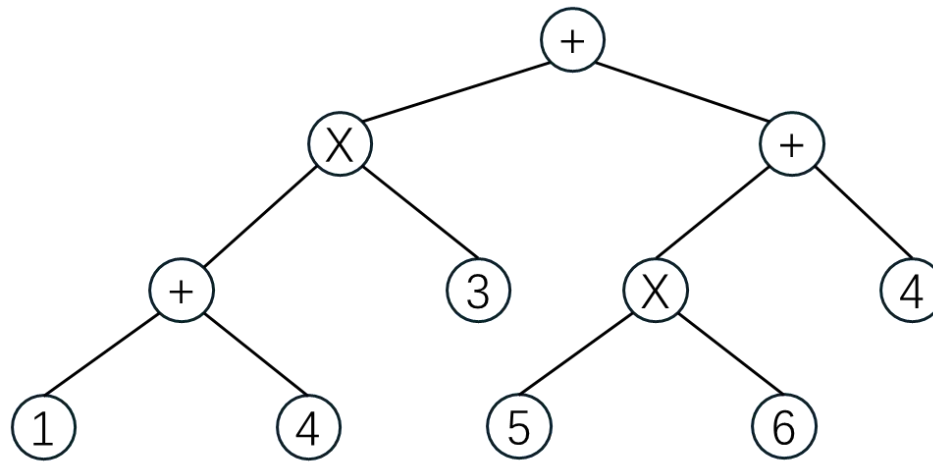
Application — Tree Deletion

- When **deleting** all the nodes in a tree,
 - if you first delete the parent, it will cause the child nodes to be inaccessible. Therefore, the parent node needs to be deleted last.
 - Therefore, it's better to choose **post-order**.



Application — Expression

- Binary trees can be used to represent algebraic expressions.
- In-order traversal is used to print the expression defined by the tree.
- Example: $((1 + 4) \times 3) + ((5 \times 6) + 4)$



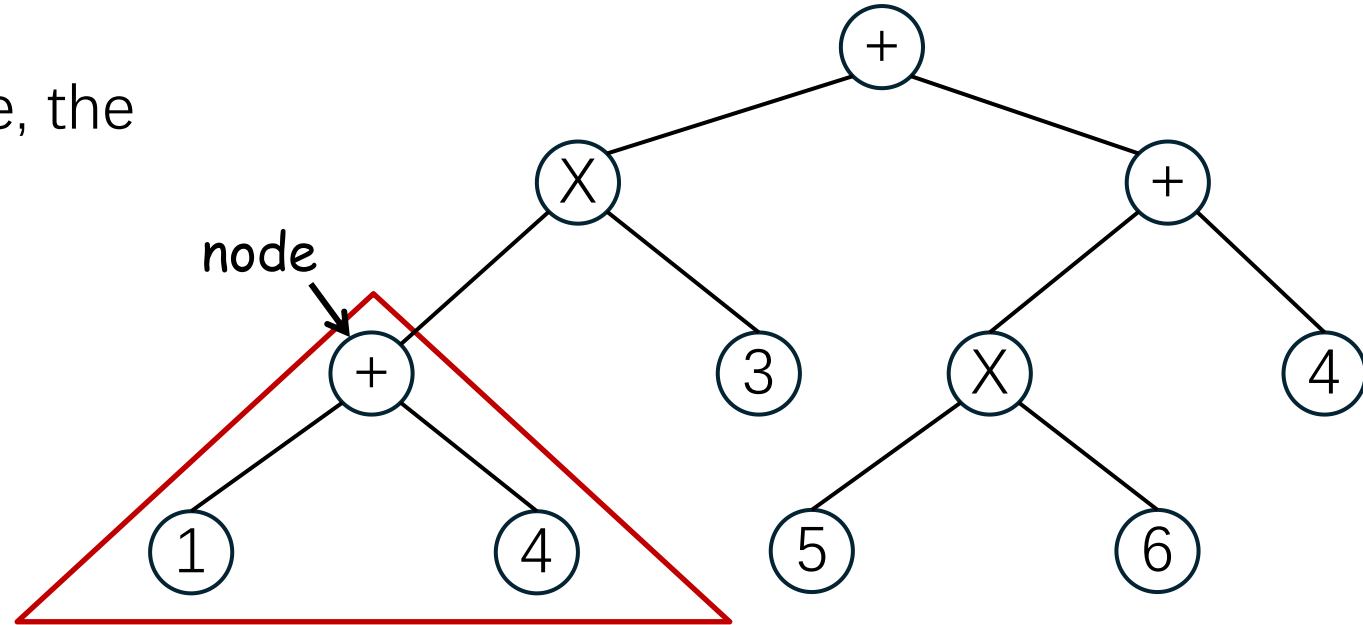
Application — Expression

- **In-order**: visit the left subtree, the root, and the right subtree.

```
def inorder(node):  
    if node is not None:  
        inorder(node.left)  
        print(node.data)  
        inorder(node.right)
```

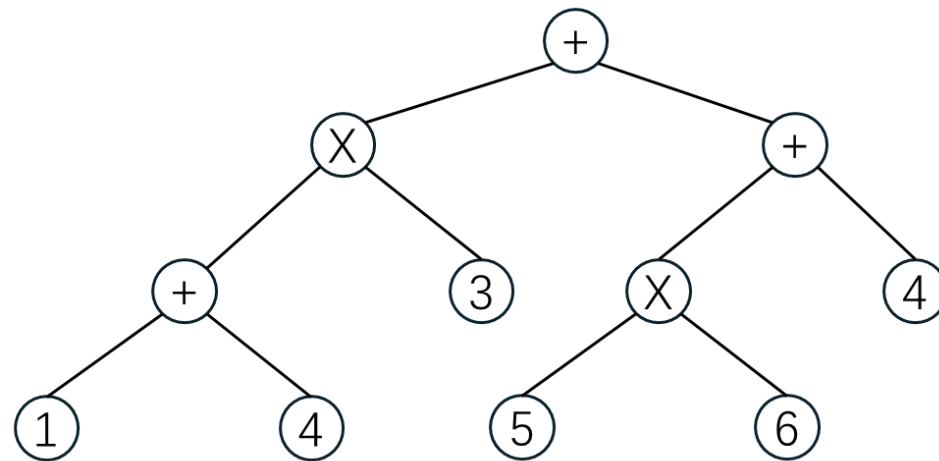
```
def inorder(node):  
    if node is not None:  
        inorder(node.left)  
        print(node.data)  
        inorder(node.right)
```

```
def inorder(node):  
    if node is not None:  
        inorder(node.left)  
        print(node.data)  
        inorder(node.right)
```



Application — Expression

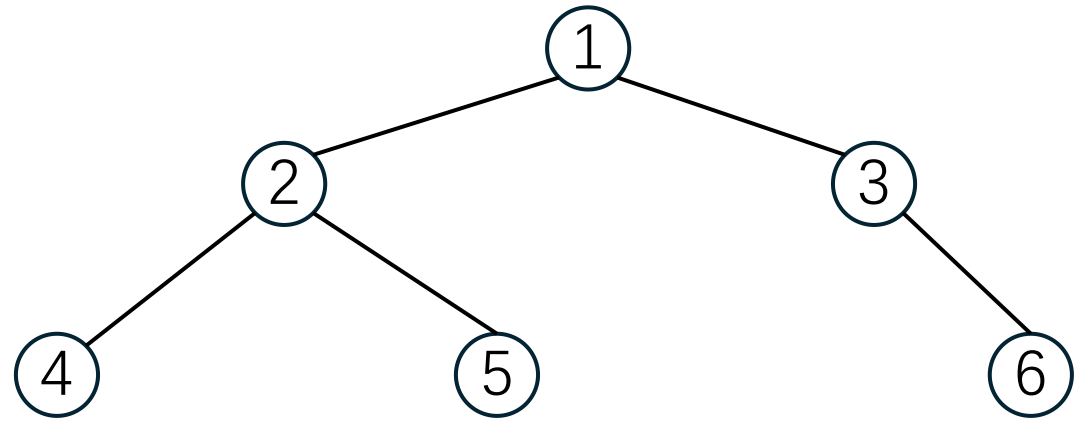
- **Post-order**: visit the left subtree, the right subtree, and the root.
- Example: $((1 + 4) \times 3) + ((5 \times 6) + 4)$.
- Post-order traversal is used to evaluate the expression (think: how?).
- Post-order: 1 4 + 3 × 5 6 × 4 + +.



Application — Counting Leaf Nodes

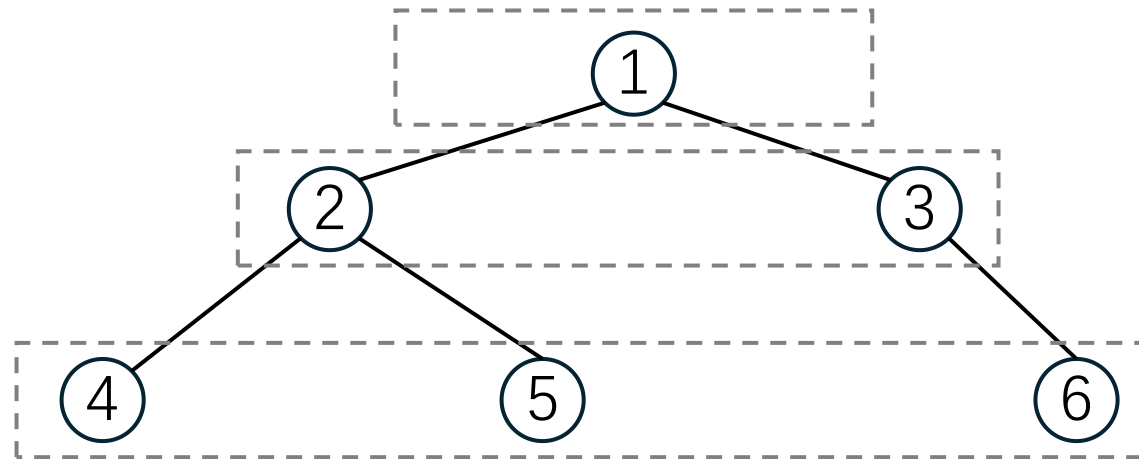
```
class Node:  
    def __init__(self, data):  
        self.left = None  
        self.right = None  
        self.data = data
```

```
def count_leaf_nodes (root):  
    if root is None:  
        return 0  
    if root.left is None and root.right is None:  
        return 1  
    return count_leaf_nodes (root.left) + count_leaf_nodes (root.right)
```



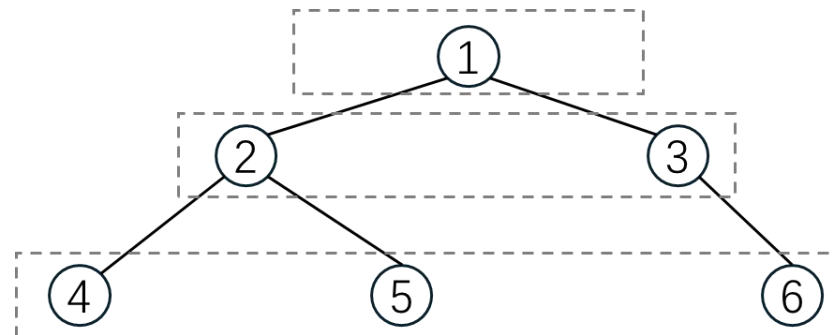
Level-order (Breadth First) Traversal

- Level-order
 - At each level, visit each node from left to right
- Traversal Order:
 - 1->2->3->4->5->6



Level-order Traversal

- Create an empty queue Q .
- Enqueue the root node of the tree to Q .
- Loop while Q is not empty:
 - Dequeue a node u from Q and visit it;
 - Enqueue the left child of u if it exists;
 - Enqueue the right child of u if it exists.



Application — Print the Value of Each Node

```
def level_order(root):  
    if root is None:  
        return []  
    q = []  
    # Enqueue Root  
    q.append(root)  
    res = []  
    while q:  
        len_q = len(q)  
        level_nodes = []  
        for _ in range(len_q):  
            node = q.pop(0)  
            level_nodes.append(node.data)  
            # Enqueue left child  
            if node.left is not None:  
                q.append(node.left)  
            # Enqueue right child  
            if node.right is not None:  
                q.append(node.right)  
        res.append(level_nodes)  
    return res
```

